



Personal Finance Tracker with Cash Flow Navigation Analytics

Group 5

Names	:	Rais Alim Fathoni	2521010074
		Mantasyaa Adini Chairunisa	2521010176
		Muhammad Alfian Pratama	2521010061

Class : 2AIDA1

PROJECT ON

Personal Financial Tracker with Cash Flow Navigation Analytics

Developed by

1. Rais Alim Fathoni
2. Muhammad Alfian Pratama
3. Mantasyaa Adini Chairunisa

Personal Finance Tracker with Cash Flow Navigation Analytics

Batch Code : 2AIDA1

Start Date : Tuesday, 26 May 2026

End Date : Monday, 15 June 2026

Name of Faculty : Harry Prasetyo Rosianto

Names of Developer :

1.Muhammad Alfian Pratama

2.Rais Alim Fathoni

3.Mantasyaa Adini Chairunisa

Date of Submission: Monday, 15 June 2026

CERTIFICATE

This is to certify that this report titled “Personal Finance Tracker with Cash Flow Navigation Analytics” embodies the original work done by Rais Alim Fathoni, Mantasyaa Adini Chairunisa, and Muhammad Alfian Pratama. Project in partial fulfilment of their course requirement at CEP-CCIT FTUI.

Coordinator:

Harry Prasetyo Rosianto

ACKNOWLEDGEMENT

The authors would like to express gratitude to Allah SWT for the health and opportunity to complete this Project 4 report titled “Personal Finance Tracker with Cash Flow Navigation Analytics.”

Sincere appreciation is given to Mr. Harry Prasetyo Rosianto, as the supervising lecturer, for guidance and valuable input during the preparation of this report. The authors also extend thanks to fellow classmates and project teammates for their support throughout the process.

The authors realize this report is not without shortcomings and therefore welcome constructive criticism and suggestions for its improvement.

BACKGROUND

Managing personal finances is a critical aspect of daily life, yet many individuals struggle to maintain consistent records of their income and expenses. Traditional methods, such as manual ledgers or basic spreadsheet tracking, are prone to human error, lack real-time analytical insights, and fail to provide adequate security for sensitive financial data. Furthermore, existing digital solutions often require tedious manual entry for every transaction, leading to user fatigue and abandonment.

Project MAP (Personal Finance Tracker) was conceptualized to solve these friction points. It is an Android-based mobile application designed to simplify financial tracking by introducing automated data entry through Natural Language Processing (NLP), robust offline-first capabilities, and deeply integrated visual analytics.

- Core Objectives

Frictionless Data Entry: Implement an NLP parser capable of understanding colloquial Indonesian transaction inputs (e.g., "beli kopi 25rb") to automatically extract the transaction type, category, and nominal amount.

Data Resilience (Offline-First): Ensure users can view and log transactions even without an internet connection by utilizing a local SQLite database (Room), which subsequently synchronizes with a cloud database (Firebase) once connectivity is restored.

Actionable Insights: Provide users with immediate, highly visual feedback on their financial health through custom-rendered charts (Doughnut, Bar, Radar, Heatmap) and calculated metrics like the "Defense Ratio" (Savings Filter).

Security & Privacy: Protect user financial data through an authentication flow requiring both an email/ password combination and a localized 6-digit security PIN.

- Target Audience & Benefits

The application targets young adults, professionals, and students who need a straightforward yet powerful tool to monitor their cash flow. The benefits are dual-fold:

For the User: Prevents premature budget depletion by setting customizable "Budget Alerts", secures financial data behind a 2FA-like PIN wall, and makes transaction logging as simple as sending a text message. The ability to export reports to PDF and CSV formats allows for external auditing or record-keeping.

For the Developer: Provides a robust architectural sandbox for implementing Modern Android Development (MAD) practices, including Jetpack Compose (Material 3), Coroutines/Flow for reactive state management, and the Repository pattern for complex data synchronization logic.

SYSTEM ANALYSIS

Project MAP strictly adheres to the Model-View-ViewModel (MVVM) architecture combined with an Offline-First approach. This structure isolates the UI from the business logic and data operations, making the app highly testable and maintainable.

View Layer (Jetpack Compose): The entire UI is built declaratively using Jetpack Compose. Screens such as `DashboardScreen` and `AnalyticsScreen` observe state changes via `Kotlin StateFlow`.

Recomposition only occurs when the underlying data changes, optimizing rendering performance.

ViewModel Layer: Classes like `TransactionViewModel` and `AuthViewModel` act as intermediaries. They handle user intents (e.g., clicking "Save Transaction") and execute background operations using Kotlin Coroutines (`viewModelScope`).

Model/Repository Layer: The `TransactionRepository` is the single source of truth. It abstracts the complexity of data fetching. When a transaction is saved, it is first written to the local Room Database. If the user is online, it simultaneously pushes the data to the Firebase Realtime Database. If offline, the

data is queued and marked as `isSynced = false`.

- "Demo Mode" Fallback Mechanism

To ensure the application remains fully functional even if Firebase configuration (`google-services.json`) is missing or server connectivity fails completely, a Demo Mode was implemented. If Firebase initialization fails, the `AuthRepository` bypasses remote authentication and injects a mock `demo_user` session. The `TransactionRepository` detects this flag and routes all CRUD operations exclusively to the local Room database, ensuring zero downtime for the user experience.

- Data Analytics & Visualization Engine

Rather than relying on heavy third-party charting libraries (like MPAndroidChart), Project MAP utilizes Jetpack Compose's native Canvas API to draw custom, performant charts. The Cash Flow Navigation system calculates three main metrics:

Savings Filter (Defense Ratio): Compares total monthly income against expenses, visualized as a linear progress bar indicating budget consumption percentage.

Hull Leak Radar: Aggregates expenses by category and plots the top 5 categories on a custom Radar Chart, helping users instantly identify where their money is "leaking."

Daily Consumption Pulse: A line chart plotting the velocity of daily expenses over a 31-day rolling window.

DATA MODEL

1. Database & Storage Architecture

Following the architecture pattern outlined in your project reference, the application relies on an offline-first data synchronization strategy. Local storage is managed via Room SQLite Database, which pairs directly with Firebase Firestore for remote cloud backups.

```
/database/  
└─ room_local.db  
    └─ users_table  
        └─ transactions_table
```

2. User Authentication Model (User)

The `User` data model represents authenticated user profiles inside the application. This model handles data segregation for multi-user safety and stores local verification records.

```
{  
  "uid": "user_987654_abc",  
  "name": "John Doe",  
  "email": "johndoe@example.com",  
  "pin": "$2a$10$eImiTX...",  
  "createdAt": 1717497276000  
}
```


Field	Type	Description
uid	String	Unique identifier generated by Firebase Authentication upon registration. Acts as the primary key.
name	String	The full name of the user used for UI personalization.
email	String	Registered email address used as login credentials. Must be unique.
pin	String	Hashed 6-digit security PIN used for quick local biometric/passcode authentication.
createdAt	Long	Unix timestamp (milliseconds) representing when the account was first created.

3. Transaction Local Model (**TransactionEntity**)

The **TransactionEntity** acts as the core database model mapping directly to the SQLite local tables via Room and documents inside Firebase Firestore.

```
{
  "id": "8f3b9a1c-4b2d-4e6f-8a7b-9c0d1e2f3a4b",
  "userId": "user_987654_abc",
  "title": "Starbucks Coffee",
  "amount": 45000.00,
  "type": "EXPENSE",
  "category": "Food & Beverage",
  "date": 1717497276000,
  "note": "Meeting with client",
  "isSynced": true
}
```

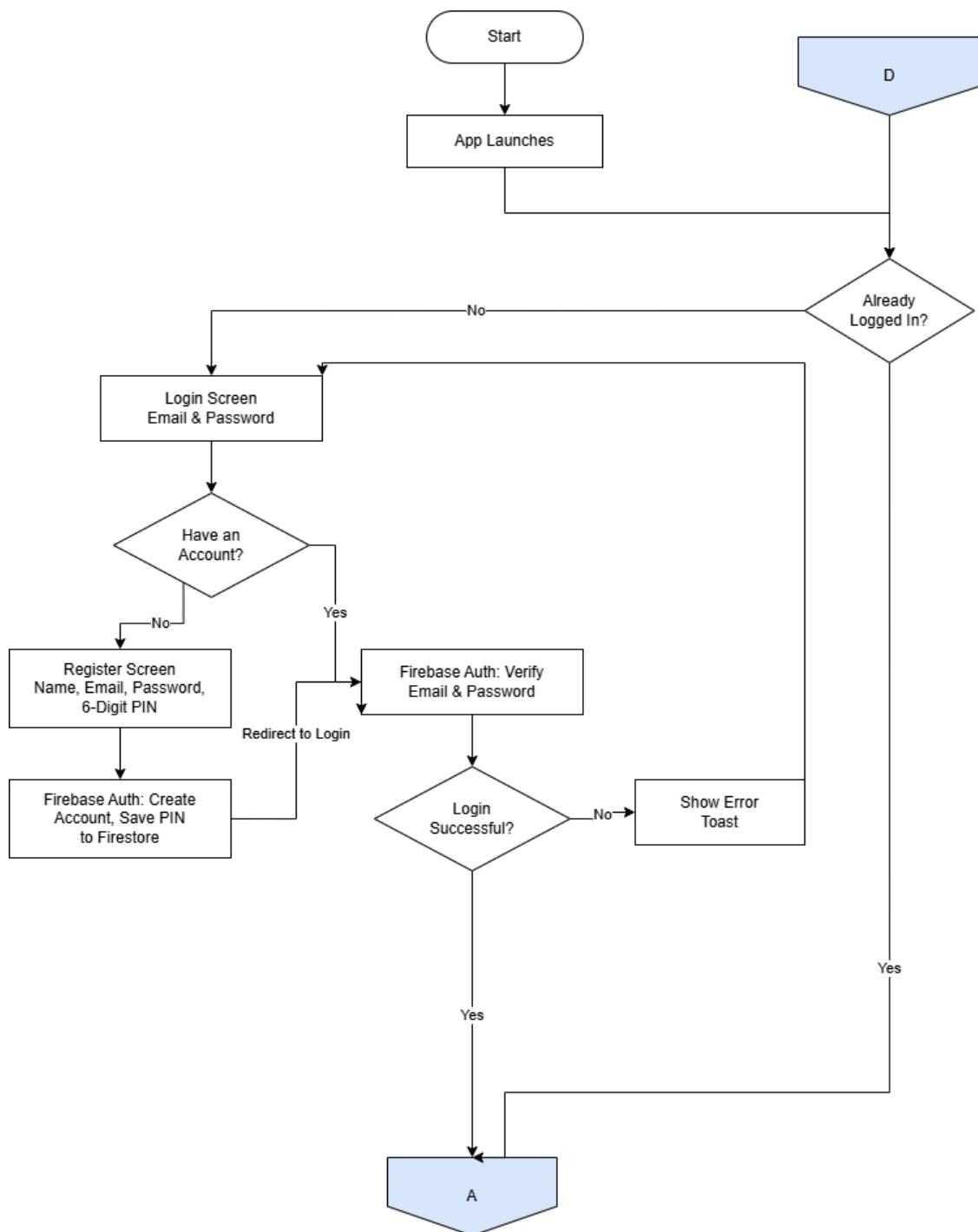
Field	Type	Description
id	String	Unique UUID string serving as the Primary Key. Prevents ID collisions during offline creation.
userId	String	Foreign Key matching the <code>User.uid</code> . Ensures transactions are isolated per user.
title	String	The main title or brief text describing the financial transaction.
amount	Double	Numerical value representing the monetary value of the transaction.
type	String	The transaction vector. Restricted strictly to INCOME or EXPENSE .
category	String	Pre-classified group identifier (e.g., Food, Transport, Salary) used for budgeting.
date	Long	Unix timestamp (milliseconds) tracking exactly when the transaction occurred.
note	String	Nullable text field used for storing extra custom descriptions or receipts.
isSynced	Boolean	Cloud sync flag tracking if local mutations have successfully updated to Firebase.

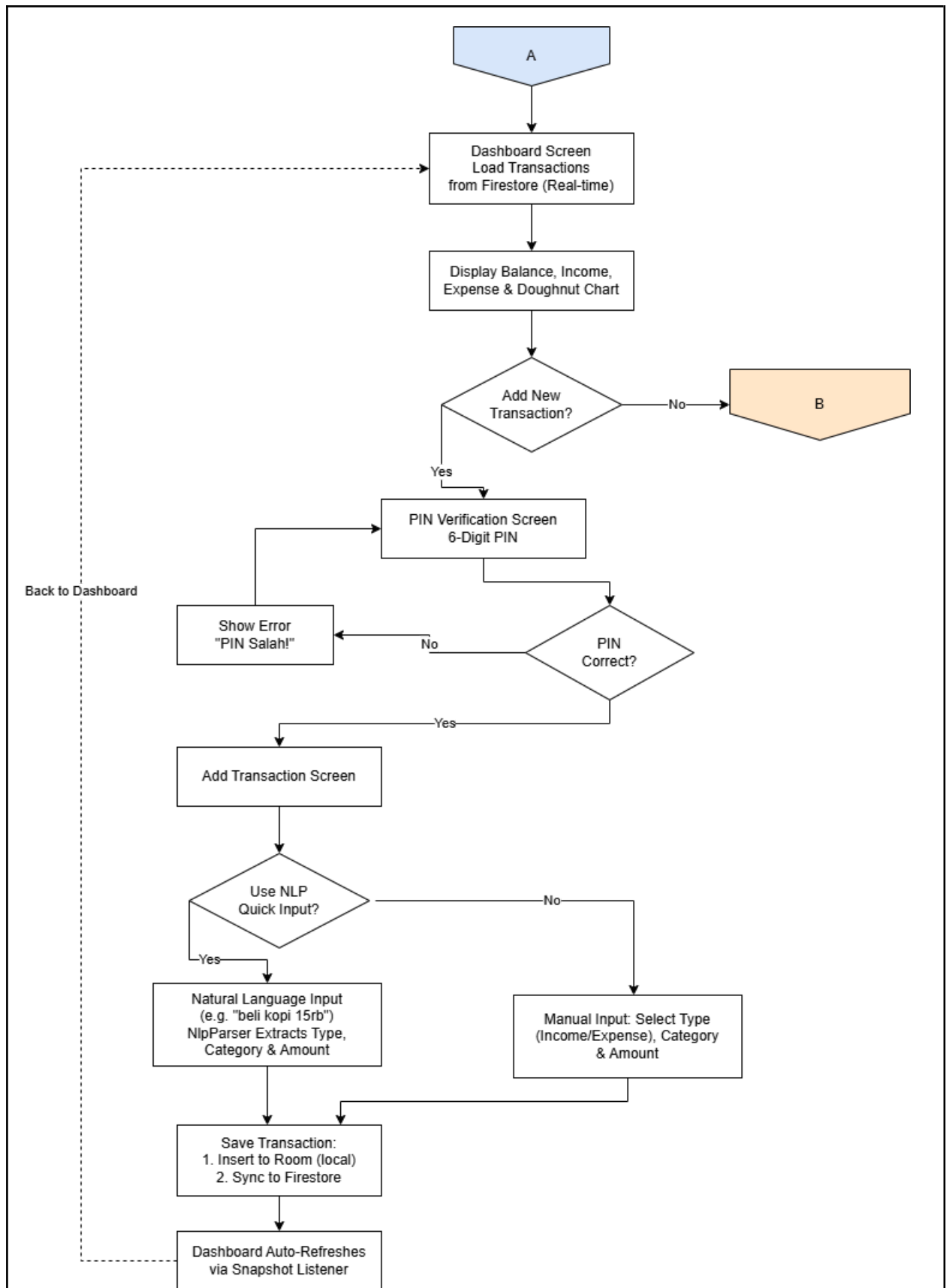
4. UI Representation (**TransactionUiEntry**)

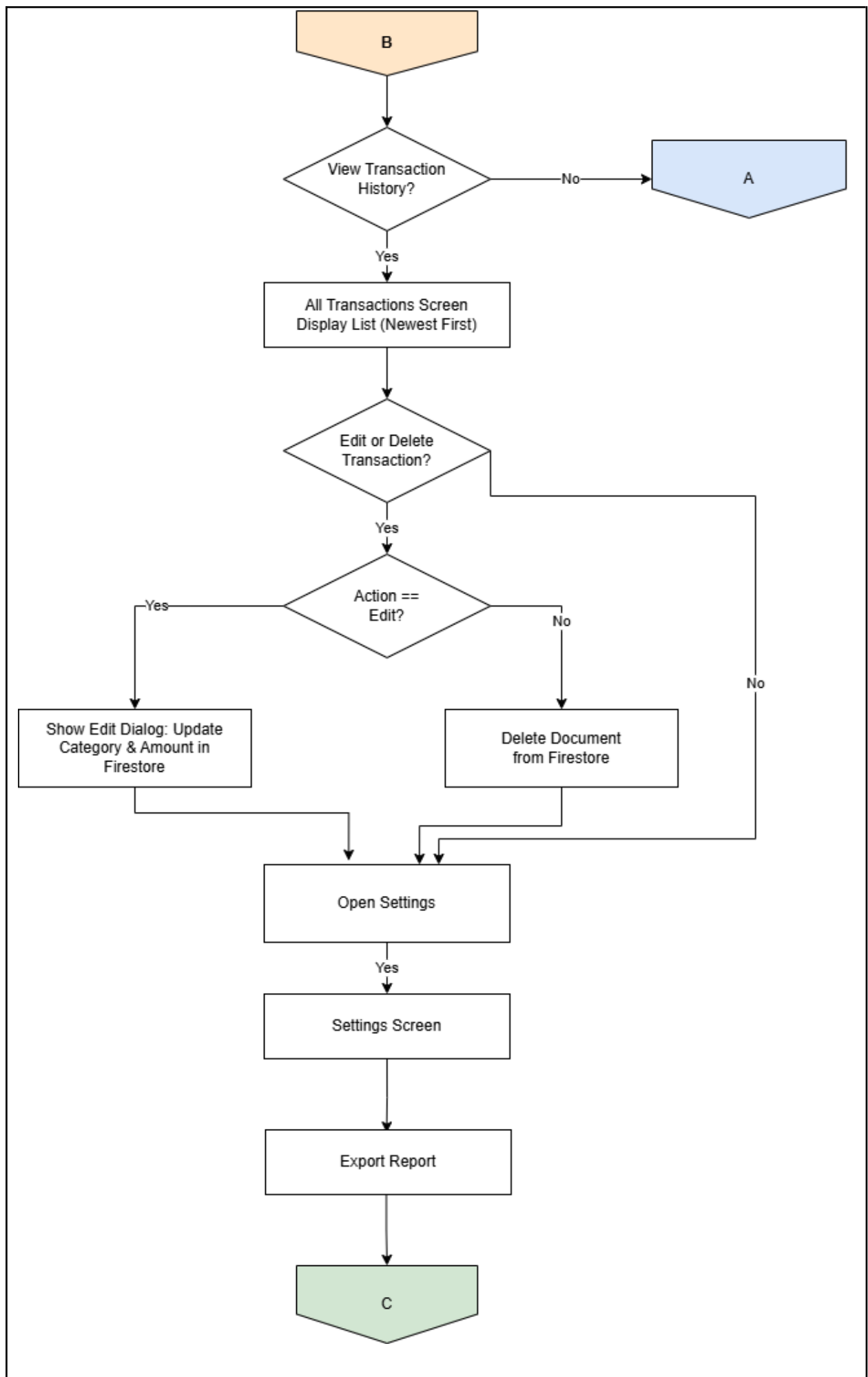
For responsive rendering across Jetpack Compose UI components, raw entity structures are converted into presentation-ready display states.

```
{
  "id": "8f3b9a1c-4b2d-4e6f-8a7b-9c0d1e2f3a4b",
  "displayAmount": "- Rp55.000",
  "displayDate": "Jun 04, 2026",
  "title": "Lunch at McDonald's",
  "category": "Food & Beverage",
  "syncStatus": "SUCCESS"
```

FLOWCHART







SOURCE CODE

1. Source Code Implementation

Before discussing the implementation details of individual application modules, it is important to note that the complete source code of the Project MAP application is organized using a modern Android application structure based on Kotlin and Jetpack Compose. Due to the size and complexity of contemporary Android projects, including every source file directly within this report would significantly reduce readability and provide limited educational value. Therefore, this section focuses on examining the most important implementation components, architectural decisions, and representative source code structures that contribute to the core functionality of the application.

The project repository follows a layered organizational approach that separates user interface components, data management utilities, application resources, and configuration files into dedicated directories. This structure promotes maintainability by ensuring that presentation logic, storage operations, and application settings remain clearly isolated from one another. Furthermore, the project leverages Jetpack Compose, Kotlin Coroutines, StateFlow, and a dual database persistence mechanism (Room and Firebase) to provide a responsive user experience and offline-first capabilities.

2. Source Code Directory Structure

Below is the directory structure of the primary source code used in the development of Project MAP, followed by an explanation of each major folder and its purpose.

ProjectMAP

```
├─ java/com/example/projectmap
│   └─ ui
│       └─ auth
│           ├── LoginFragment.kt
│           ├── RegisterFragment.kt
│           └─ PinVerifyFragment.kt
```

- | | └ dashboard
 - | | | └ DashboardFragment.kt
 - | | | └ AddTransactionFragment.kt
 - | | | └ AllTransactionsFragment.kt
 - | | | └ TransactionViewModel.kt
- | | └ analytics
 - | | | └ AnalyticsFragment.kt
- | | └ settings
 - | | | └ SettingsFragment.kt
- | └ util
- | | └ MainActivity.kt
- | | └ ProjectMapApp.kt
- | |
- └ res
 - └ drawable
 - | | └ bg_outlined_input.xml
 - | | └ bg_rounded_btn_nlp.xml
 - | | └ bg_category_selected.xml
 - | | └ bg_category_unselected.xml
 - | | └ bg_rounded_btn_save.xml
 - └ layout
 - | | └ activity_main.xml
 - | | └ fragment_dashboard.xml
 - | | └ fragment_analytics.xml
 - | | └ fragment_settings.xml
 - | | └ fragment_add_transaction.xml

| | └─ fragment_all_transactions.xml

| |

└─ AndroidManifest.xml

2.1 Folder Explanations

- `com/example/projectmap/`: The primary application package containing all Kotlin source files responsible for implementing application behavior, user interaction, and data processing.
 - `ProjectMapApp.kt` & `MainActivity.kt`: Acts as the central entry point of the application, managing Firebase initialization, global application state, screen navigation, and theme application.
 - `data/`: Contains the core infrastructure layer responsible for handling persistence and remote synchronization.
 - `local/`: Manages the Room SQLite database, including entities and Data Access Objects (DAOs).
 - `repository/`: Contains the `AuthRepository` and `TransactionRepository` which wrap Firebase and local database calls, managing the offline-first sync logic.
 - `navigation/`: Houses `Routes.kt`, which defines type-safe routing objects to eliminate string-based navigation typos.
 - `ui/`: Contains all user interface implementation files developed using Jetpack Compose, organized by feature (auth, dashboard, analytics, etc.).
 - `components/`: Reusable interface elements, notably `Charts.kt` which handles custom Canvas-based data visualization.
 - `util/`: Contains isolated utility classes for NLP processing (`NlpParser.kt`), document generation (`ExportUtils.kt`), and system notifications (`NotificationUtils.kt`).

3. Core Implementation Details

3.1 Application Initialization & Navigation

One of the most important implementation components is `MainActivity.kt`, which functions as the central orchestration layer for coordinating navigation. Unlike traditional Android applications that rely on multiple Activities, Project MAP adopts a modern single-activity design built entirely using Jetpack Compose via the `setContent` block.

The navigation system is implemented using the Navigation Compose component and a sealed interface named `Route`. Using a sealed interface instead of plain strings prevents typos in navigation calls and allows the compiler to catch missing route cases at build time. The `NavHost` checks the current Firebase session on startup to dynamically route users either to the login screen or directly to the dashboard.

```
// MainActivity.kt (Abridged)
package com.example.projectmap

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.runtime.*
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.projectmap.ui.auth.LoginScreen
import com.example.projectmap.ui.dashboard.DashboardScreen
import com.example.projectmap.ui.theme.ProjectMAPTheme
import com.google.firebase.auth.FirebaseAuth

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
```

```

        val systemTheme = isSystemInDarkTheme()

        var isDarkMode by remember { mutableStateOf(systemTheme) }

        ProjectMAPTheme(darkTheme = isDarkMode) {

            val navController = rememberNavController()

            val currentUser = remember {
                FirebaseAuth.getInstance().currentUser }

            val startDestination = if (currentUser != null) "dashboard"
            else "login"

            NavHost(navController = navController, startDestination =
startDestination) {

                composable("login") {

                    LoginScreen(

                        onLoginSuccess = {
                            navController.navigate("dashboard") { popUpTo("login") { inclusive = true } }
                        },

                        onNavigateToRegister = {
                            navController.navigate("register") }

                    )

                }

                composable("dashboard") {

                    DashboardScreen(

                        isDarkMode = isDarkMode,

                        onThemeToggle = { isDarkMode = !isDarkMode },

                        onNavigateToAddTransaction = {
                            navController.navigate("pin_verify") },

                        onNavigateToAllTransactions = {
                            navController.navigate("all_transactions") },

                        onNavigateToAnalytics = {
                            navController.navigate("analytics") { popUpTo(0) { inclusive = true } } },

                        onNavigateToSettings = {
                            navController.navigate("settings") { popUpTo(0) { inclusive = true } } }

                    )

                }

            }

        }
    }
}

```

```

        // Other composable routes...

    }

}

}

}

```

```

// Routes.kt

package com.example.projectmap.navigation

import kotlinx.serialization.Serializable

sealed interface Route {

    @Serializable data object Login : Route

    @Serializable data object Register : Route

    @Serializable data object PinSetup : Route

    @Serializable data object PinVerify : Route

    @Serializable data object Dashboard : Route

    @Serializable data object Analytics : Route

    @Serializable data object Settings : Route

}

```

3.2 Offline-First Synchronization Architecture

The persistence subsystem is primarily implemented through `TransactionRepository.kt`, which acts as the heart of the offline-first architecture. Rather than forcing the user to wait for cloud confirmation, operations are handled locally first using Room (`TransactionDao`).

When a user adds a transaction, `addTransaction` always writes to the local database first, guaranteeing that the data is immediately visible in the UI even with no internet connection. Subsequently, the repository attempts to push the data to Firebase via `syncToFirebase`.

```
// TransactionRepository.kt (Abridged)
```

```
package com.example.projectmap.data.repository
```

```
import com.example.projectmap.data.local.TransactionDao
```

```
import com.example.projectmap.data.local.TransactionEntity
```

```
import com.google.firebase.database.FirebaseDatabase
```

```
import kotlinx.coroutines.flow.Flow
```

```
import kotlinx.coroutines.tasks.await
```

```
class TransactionRepository(
```

```
    private val transactionDao: TransactionDao,
```

```
    private val userId: String,
```

```
    private val isDemoMode: Boolean = false
```

```
) {
```

```
    private val database = if (!isDemoMode) {
```

```
        try {
```

```
        FirebaseDatabase.getInstance().getReference("transactions").child(userId) }
```

```
        catch (e: Exception) { null }
```

```
    } else null
```

```
    val allTransactions: Flow<List<TransactionEntity>> =  
    transactionDao.getAllTransactions(userId)
```

```
    suspend fun addTransaction(transaction: TransactionEntity) {
```

```
        // 1. Save locally first
```

```
        transactionDao.insertTransaction(transaction)
```

```

        // 2. Then push to cloud
        if (!isDemoMode) {
            syncToFirebase(transaction)
        }
    }

    private suspend fun syncToFirebase(transaction: TransactionEntity) {
        if (database == null) return
        try {
            val ref = if (transaction.firebaseId.isEmpty()) database.push()
            else database.child(transaction.firebaseId)

            val firebaseId = ref.key ?: ""

            val updatedTransaction = transaction.copy(firebaseId = firebaseId,
            isSynced = true)

            ref.setValue(updatedTransaction).await()

            // Mark as synced in Room
            transactionDao.updateTransaction(updatedTransaction)
        } catch (e: Exception) {
            e.printStackTrace() // Keep isSynced = false, retry later
        }
    }
}

// TransactionDao.kt
package com.example.projectmap.data.local

import androidx.room.*
import kotlinx.coroutines.flow.Flow

```



```
@Dao
```

```
interface TransactionDao {
```

```
    @Query("SELECT * FROM financial_transactions WHERE userId = :userId ORDER  
BY date DESC")
```

```
    fun getAllTransactions(userId: String): Flow<List<TransactionEntity>>
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertTransaction(transaction: TransactionEntity)
```

```
    @Update
```

```
    suspend fun updateTransaction(transaction: TransactionEntity)
```

```
    @Delete
```

```
    suspend fun deleteTransaction(transaction: TransactionEntity)
```

```
    @Query("SELECT * FROM financial_transactions WHERE isSynced = 0")
```

```
    suspend fun getUnsyncedTransactions(): List<TransactionEntity>
```

```
}
```

3.3 Natural Language Processing (NLP) Engine

To facilitate frictionless data entry, the `NlpParser.kt` object provides an on-device Indonesian natural language parser. It requires no internet connection and executes almost instantaneously.

The engine uses regular expressions, specifically `(\d+)\s*(rb|ribu|jt|juta)?`, to capture numeric values followed by optional Indonesian currency suffixes. The parser then multiplies the base number by 1,000 for "rb/ribu" or 1,000,000 for "jt/juta". Following amount extraction, category classification uses a priority chain of `if-else if` statements matching against lists of predefined keywords to automatically categorize the transaction.

```
// NlpParser.kt
```

```
package com.example.projectmap.util
```

```

object NlpParser {

    data class ParsedTransaction(val type: String, val category: String, val
amount: Int)

    fun parseNaturalLanguage(input: String): ParsedTransaction? {

        val lowerInput = input.lowercase().trim()

        var amount = 0

        val numberRegex = Regex("(\\d+)\\s*(rb|ribu|jt|juta)?")
        val matchResult = numberRegex.find(lowerInput)

        if (matchResult != null) {

            val baseNumber = matchResult.groupValues[1].toInt()

            val multiplierStr = matchResult.groupValues[2]

            amount = when (multiplierStr) {

                "rb", "ribu" -> baseNumber * 1000

                "jt", "juta" -> baseNumber * 1000000

                else -> baseNumber

            }

        } else { return null }

        var type = "expense"

        var category = "Lainnya"

        val incomeKeywords = listOf("gaji", "gajian", "bonus", "dapat",
"masuk")

        val foodKeywords = listOf("makan", "minum", "beli kopi", "kopi",
"warteg", "resto", "makanan")

        if (incomeKeywords.any { lowerInput.contains(it) }) {

            type = "income"

```

```

        category = "Pemasukan"

    }

    else if (foodKeywords.any { lowerInput.contains(it) }) { category =
"Makanan" }

    // Other categories...

    return ParsedTransaction(type, category, amount)

}
}

```

3.4 Data Visualization Systems

Data visualization is handled by `Charts.kt`, which contains reusable chart composables drawn entirely with Compose Canvas, eliminating the need for bulky third-party charting libraries.

// Charts.kt (Abridged)

```

package com.example.projectmap.ui.components

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.*
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.unit.dp

```

```

@Composable

fun DoughnutChart(data: Map<String, Long>, modifier: Modifier =
Modifier) {

    val total = data.values.sum().toFloat()

    val colors = listOf(

        MaterialTheme.colorScheme.primary,
MaterialTheme.colorScheme.secondary,

        MaterialTheme.colorScheme.tertiary,
MaterialTheme.colorScheme.error,

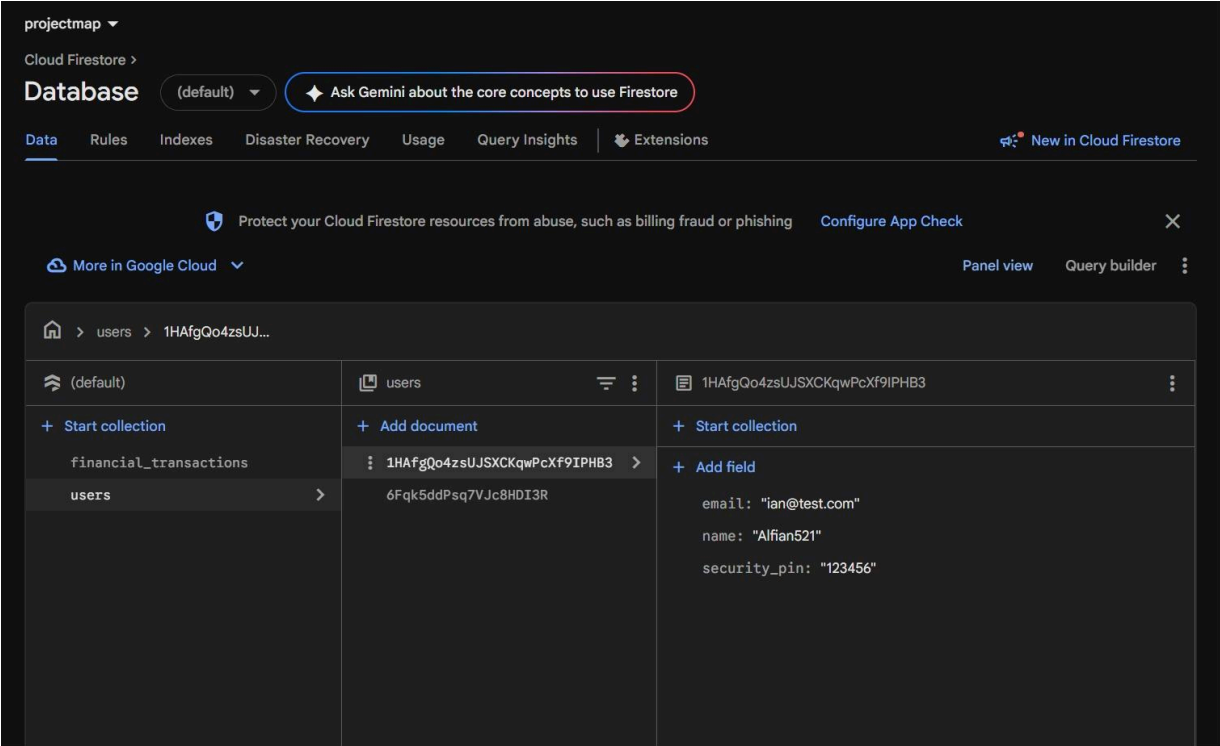
        MaterialTheme.colorScheme.primaryContainer,
MaterialTheme.colorScheme.secondaryContainer

    )

    Box(modifier = modifier, contentAlignment = Alignment.Center) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            var startAngle = -90f
            data.values.forEachIndexed { index, value ->
                val sweepAngle = (value / total) * 360f
                drawArc(
                    color = colors.getOrElse(index) { Color.Gray },
                    startAngle = startAngle,
                    sweepAngle = sweepAngle,
                    useCenter = false,
                    style = Stroke(width = 40.dp.toPx(), cap =
StrokeCap.Round)
                )
                startAngle += sweepAngle
            }
        }
    }
}

```

DATABASE



projectmap

Cloud Firestore

Database

(default)

Ask Gemini about the core concepts to use Firestore

Data

Rules

Indexes

Disaster Recovery

Usage

Query Insights

Extensions

New in Cloud Firestore

Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing

Configure App Check

More in Google Cloud

Panel view

Query builder

financial_transa... > 2635XKZD2scT...

(default)	financial_transactions	2635XKZD2scTGDtIDjfo
+ Start collection	+ Add document	+ Start collection
financial_transactions >	2635XKZD2scTGDtIDjfo >	+ Add field
users	2Ztn22SY0DVjhFZlqmm 3GMZn9xmo6aK1rjRuNr0 4tyKo8XutKX0A6WuMxMz 53E6W969xge9Uwuf5Ppg 7269MCs3gySParC1hHLV 8Vy6MJzVX5kbMQmYbJsL A3304ru2NSCpjLx1JuZx Bq8nQvSsG3nGd98jLJsY	amount_of_money: 300000 category: "Makanan" date: June 2, 2026 at 8:38:35 AM UTC+7 transaction_type: "expense" user_id: "PW8EJA3VivMFSjOcQ35MKyZHps02"

projectmap

Cloud Firestore

Database

(default)

Ask Gemini about the core concepts to use Firestore

Data

Rules

Indexes

Disaster Recovery

Usage

Query Insights

Extensions

New in Cloud Firestore

Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing

Configure App Check

More in Google Cloud

Panel view

Query builder

users > 6Fqk5ddPsq7VJ...

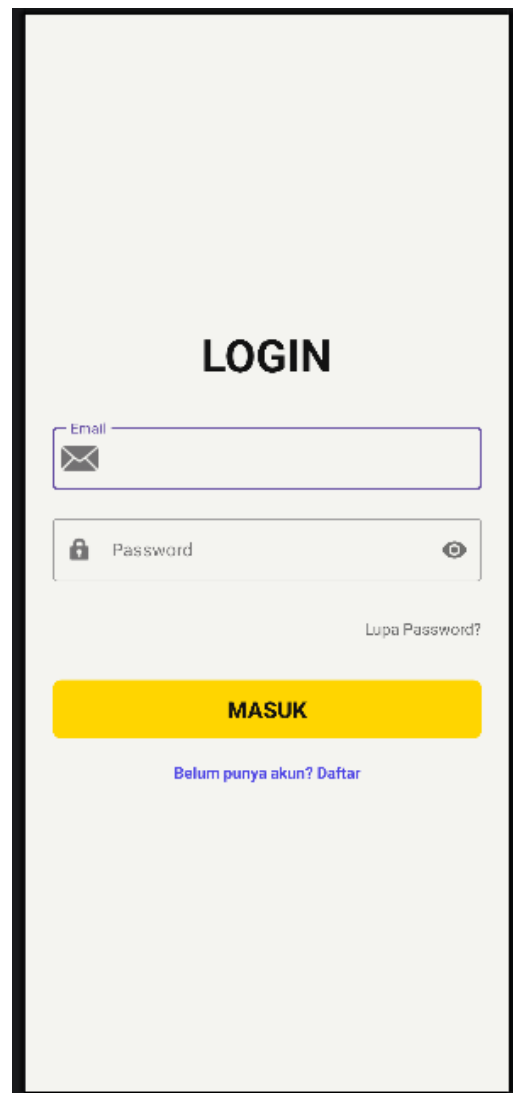
(default)	users	6Fqk5ddPsq7VJc8HDI3R
+ Start collection	+ Add document	+ Start collection
financial_transactions	1HAFgQo4zsUJSXCKqwPcXF9IPHB3	+ Add field
users >	6Fqk5ddPsq7VJc8HDI3R >	email: "alfian25@test.com" name: "Iyan9" security_pin: "123456"
	PW8EJA3VivMFSjOcQ35MKyZHps02 d481iL7bCSYfILKfUC5k80BgCpt2 zJQ0kEoKS4o4iivEaMU7	

DESIGN INTERFACE

1. Login Screen

The program output displays the authentication entry point of the Financial Tracker application, where users begin their session by providing their registered credentials. The screen presents a clean, minimalist layout built on an off white Neo Brutalism background. At the center of the screen, the title "**LOGIN**" is displayed in large, bold typography to clearly communicate the screen's purpose.

Below the title, two input fields are presented: an **Email** field with an envelope icon on the left, and a **Password** field with a lock icon and an eye toggle button on the right that allows users to show or hide their password. A "**Lupa Password?**" link is aligned to the right below the password field, which when tapped opens an in-screen dialog allowing the user to submit their email and receive a Firebase password reset link.

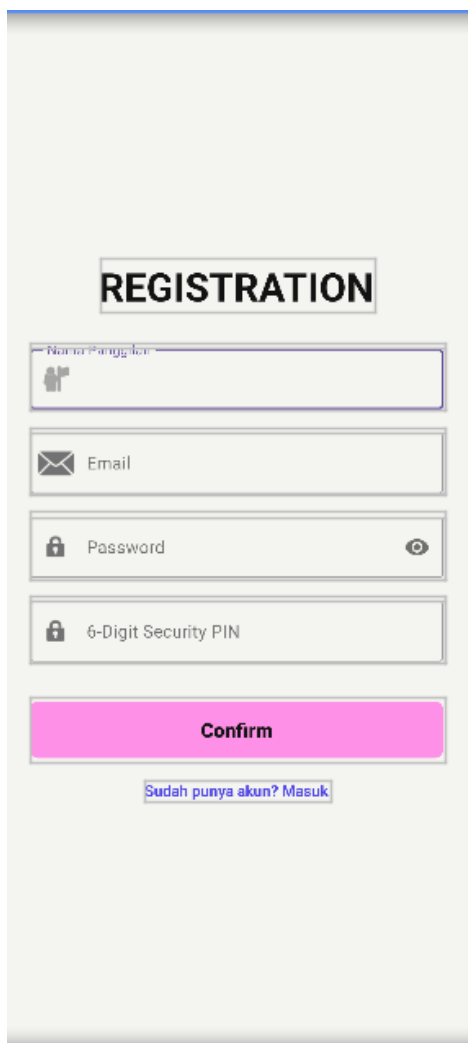
A mockup of a login screen with a light gray background. At the top center is the word "LOGIN" in bold black capital letters. Below it are two input fields. The first field is labeled "Email" in small blue text and has an envelope icon on the left. The second field is labeled "Password" in small gray text and has a lock icon on the left and an eye icon on the right. To the right of the password field is a link that says "Lupa Password?" in small gray text. Below the input fields is a large yellow button with the word "MASUK" in bold black capital letters. At the bottom of the screen is a link that says "Belum punya akun? Daftar" in small blue text.

The main action is performed through the "**MASUK**" button, rendered in bold yellow with a hard black border and drop shadow following the Neo Brutalism design. While authentication is in progress, the button is disabled and replaced with a loading spinner to prevent duplicate submissions. At the bottom of the screen, a "**Belum punya akun? Daftar**" link navigates the

user to the Registration screen. This output allows users to securely authenticate into the application using their Firebase email and password credentials.

2. Registration Screen

The program output displays the new account creation interface, which allows first-time users to register and set up their Financial Tracker account. The screen title "**REGISTRATION**" is displayed prominently at the top in bold uppercase text, clearly indicating the screen's function.

A screenshot of a mobile application's registration screen. At the top, the word "REGISTRATION" is displayed in bold, black, uppercase letters. Below the title, there are four input fields arranged vertically. The first field is labeled "Nama Panggilan" (Display Name) and features a person icon. The second field is labeled "Email" and features an envelope icon. The third field is labeled "Password" and features a lock icon and a visibility toggle (an eye icon). The fourth field is labeled "6-Digit Security PIN" and features a lock icon. Below the input fields is a large, pink button with the word "Confirm" in bold, black, uppercase letters. At the bottom of the screen, there is a link that says "Sudah punya akun? Masuk" (Already have an account? Login) in blue text.

The registration form consists of four input fields arranged vertically. The first field, "**Nama Panggilan**", uses a person icon and accepts the user's display name. The second field accepts the user's **Email** address with an envelope icon. The third field accepts a **Password** with a lock icon and a visibility toggle, enforcing a minimum of 6 characters as required by Firebase Authentication. The fourth and final field accepts a **6 Digit Security PIN** this PIN is masked with dots as the user types and is limited to exactly 6 digits through input validation, enforcing no more characters than allowed.

At the bottom of the form, the "**Confirm**" button is styled in bold pink with a hard black border, consistent with the Neo Brutalism design language. While the registration process is running, a circular loading indicator replaces the button label to communicate progress. A "**Sudah punya akun? Masuk**" link is provided below for users who navigated here by mistake. Upon successful registration, the user's account is created in Firebase Authentication, their display name is saved to their Firebase profile, and their security PIN is stored in Cloud Firestore under their user document. The app then signs the user out and redirects them to the Login screen to reauthenticate.

3. Dashboard Screen

The program output displays the main financial overview interface, which serves as the central hub of the Financial Tracker application after the user successfully logs in. At the top of the screen, a personalized greeting is displayed "**Selamat Siang, Iyan9!**" adapting the salutation to the time of day and showing the user's registered display name to confirm they are logged into the correct account.



Below the greeting, three filter buttons "**Hari Ini**", "**Bulan Ini**", and "**Semua**" allow the user to filter the transaction view by time period. A "**Batas Peringatan: 80%**" indicator with an "**Ubah Batas**" link is shown just below the filter row, informing the user of their currently configured spending alert threshold.

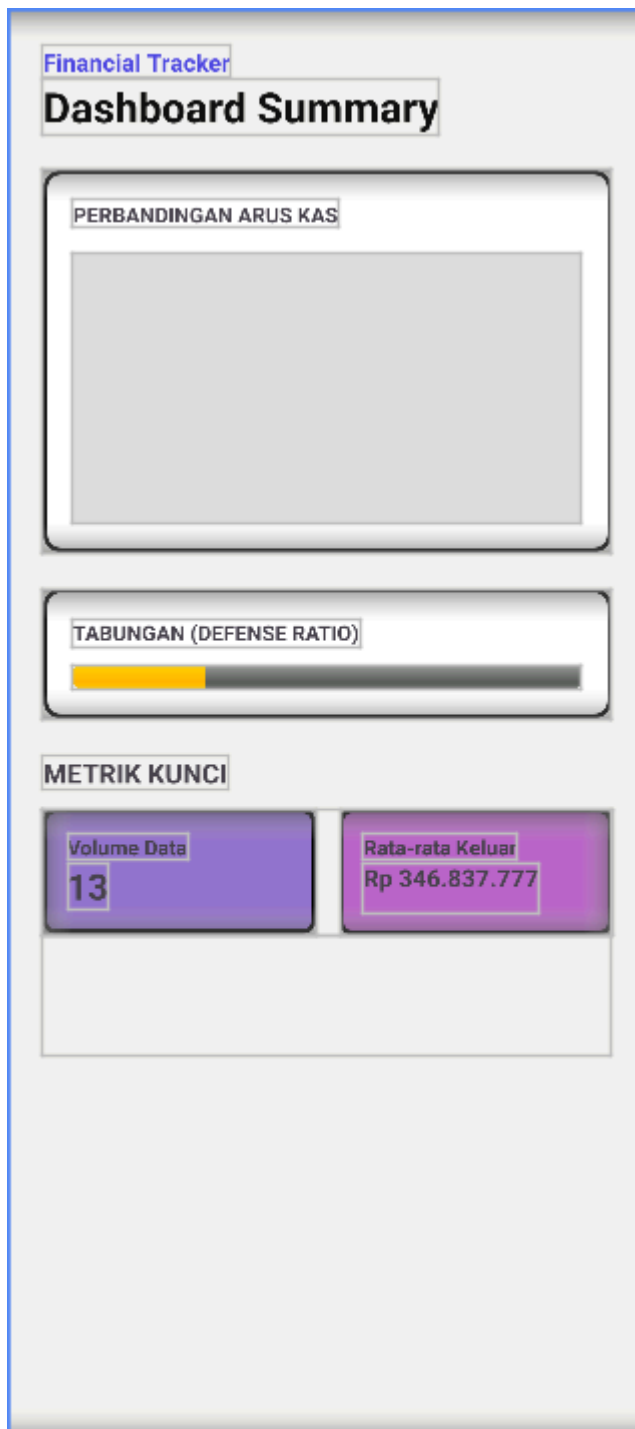
The screen then presents three summary cards: the prominent "**TOTAL SALDO**" card, displayed in a bold indigo blue background showing the current balance of **Rp 9.83 M**, followed by two side by side cards for "**PEMASUKAN**" showing **Rp 10.75 M** and "**PENGELUARAN**" showing **Rp 3.12 M**. These values are calculated in real time from the Firestore transaction collection via a snapshot listener.

Below the summary cards, an "**ALOKASI**" chart section displays a doughnut chart visualizing the spending distribution by category, giving the user an immediate visual breakdown of where their money is going. At

the bottom of the screen, the "**TRANSAKSI TERAKHIR**" list displays the most recent

transaction entries sorted from newest to oldest, providing a quick glance at recent financial activity without navigating away from the dashboard.

4. Analytics Screen



The program output displays the financial analytics interface, which provides the user with a comprehensive data-driven visual analysis of their financial activity. The screen is titled **"Dashboard Summary"** under the **"Financial Tracker"** label at the top.

The first section, **"PERBANDINGAN ARUS KAS"**, presents a bar chart that visually contrasts total income against total expenses using two distinctly colored bars green for income and red for expenses. This allows the user to immediately see whether their financial position is healthy or whether expenses are outpacing income.

The second section, **"TABUNGAN (DEFENSE RATIO)"**, displays a horizontal linear progress bar that shows the proportion of income that has been spent. The bar fills from left to right, colored yellow at low spending levels and shifting to red as spending approaches or exceeds income. A percentage label and a **"Sisa Tabungan"** amount are shown beside the bar. This metric helps the user

understand at a glance how much financial buffer they still have for the current period.

The third section, **"METRIK KUNCI"**, presents two summary cards side by side. The left card, **"Volume Data"**, displays the total number of transactions recorded in this output, **13** transactions. The right card, **"Rata-rata Keluar"**, shows the average expense amount per transaction, displayed as **Rp 346,837,777** in this output. Both cards use a purple background color consistent with the app's Material 3 secondary color scheme. All data shown in this screen is derived from the same real time Firestore snapshot listener used by the Dashboard, ensuring the analytics are always synchronized with the latest transaction data.

5. Settings Screen

The program output displays the account settings and configuration interface of the Financial Tracker application. The screen is titled **"PENGATURAN AKUN"** in large bold uppercase typography, with the **"Financial Tracker"** label and a dark/light mode toggle icon positioned at the top right.

The first section is the **Profile Card**, which displays the user's account information. A circular avatar with the user's initial letter in this output showing **"I"** for Iyan9 is displayed on the left side in the app's indigo primary color. To the right of the avatar, the user's display name **"Iyan9"** and their registered email address **"alfian25@test.com"** are shown. This section is enclosed in a card with a hard black border, consistent with the Neo Brutalism design.

The second section is the **"EKSPOR LAPORAN"** card, which contains two export buttons side by side: **"Unduh CSV"** in green and **"Unduh PDF"** in red. Tapping either button opens the Android system file picker,

allowing the user to choose where to save the exported report on their device. The CSV export



generates a structured spreadsheet compatible file with columns for date, category, type, and amount. The PDF export generates a formatted A4 document with a report title and a table of up to 35 most recent transactions.

Below the export card, two action buttons are presented. The **"GANTI PIN KEAMANAN"** button opens an in screen dialog where the user can enter a new 6 digit PIN, which is then updated in their Firestore user document. The **"KELUAR AKUN"** button, displayed with a black background and white text, calls Firebase Authentication's `signOut()` method, clears the session, and redirects the user back to the Login screen. All buttons follow the Neo Brutalism styling with hard black borders and offset drop shadows.

CONFIGURATION

Hardware : Lenovo LOQ

Operating System : Windows 11 64-bit

Software : Android Studio Panda 4 | 2025.3.4 Patch 1

Firestore SDK version : Firestore BOM 34.13.0

JDK version : OpenJDK 64-Bit Server VM by JetBrains s.r.o.

Gradle version : gradle-9.4.1

Gradle plugin : 9.2.1

Target SDK :

- Compiled SDK: 37
- Target SDK: 37
- Minimum SDK: 24

PROJECT FILE DETAIL

No.	File Name	Remarks
1	Personal Financial Tracker with Cash Flow Navigation Analytic .PDF	Mantasya, Rais, Alfian

The source repository for the project is available through GitHub:

https://github.com/Mapzoll/PersonalFinancialTracker_MobileApp.git